

Documentação Técnica: Game Wishlist Tracker

Data: 22 de Junho de 2026

1. Visão Geral do Projeto

O **Game Wishlist Tracker** é uma aplicação web full-stack desenvolvida para monitorar preços de jogos desejados pelo usuário (como Hogwarts Legacy ou Phasmophobia, por exemplo). O sistema permite o cadastro de jogos, definição de preços-alvo e visualização em uma interface moderna e responsiva.

2. Stack Tecnológica

Camada	Tecnologia	Justificativa
Front-end / Back-end	Next.js (App Router) + React	Renderização híbrida, rotas de API nativas e facilidade de deploy.
Linguagem	TypeScript	Segurança de tipagem e prevenção de erros em tempo de desenvolvimento.
Estilização	Tailwind CSS	Desenvolvimento ágil de UI com classes utilitárias.
Banco de Dados	Supabase (PostgreSQL)	Banco de dados relacional robusto com API instantânea.

Camada	Tecnologia	Justificativa
Hospedagem	Vercel	Integração nativa com Next.js e CI/CD automatizado.

3. Estrutura de Diretórios (Padrão Enterprise)

O projeto adota o isolamento do código-fonte utilizando o diretório `src/` para separar as lógicas da aplicação das configurações da raiz, padrão amplamente adotado pelo mercado.

```
/game-wishlist-tracker
├── /src
│   ├── /app          # Rotas e páginas (Next.js App Router)
│   ├── /components   # Componentes React reutilizáveis
│   ├── /lib          # Configurações de bibliotecas (ex: cliente Supabase)
│   └── /types        # Definições de interfaces TypeScript
├── .env.local        # Variáveis de ambiente sensíveis
├── tailwind.config.ts # Configuração de estilo
└── tsconfig.json     # Configuração do TypeScript
```

4. Estrutura de Banco de Dados (Schema)

A tabela principal armazenará as informações dos jogos monitorados.

- **Tabela:** wishlist
- **Colunas:**
 - id (UUID, Primary Key) - Identificador único do registro.
 - game_title (VARCHAR) - Nome do jogo.
 - target_price (DECIMAL) - Preço desejado pelo usuário.
 - current_price (DECIMAL) - Preço atual no mercado.
 - created_at (TIMESTAMPTZ) - Data de inserção no sistema.

5. Convenções de Desenvolvimento e Qualidade

1. **Commits:** Utilizar Conventional Commits (ex: feat: adiciona tabela supabase, fix: corrige botão de salvar) para manter o histórico do Git limpo e rastreável.
2. **Segurança:** Chaves de API e strings de conexão de banco de dados devem residir exclusivamente no arquivo .env.local e ser declaradas no .gitignore.
3. **Componentização:** Componentes de UI genéricos (botões, inputs, modais) devem ser isolados na pasta src/components para reuso em toda a aplicação.